

Overall Architecture

The system is built around a real-time audio bridge between Twilio and Google Gemini Live. Twilio places an outbound call to the hospital agent's phone number, and once the call connects, the audio is streamed bidirectionally over a Twilio Media Stream WebSocket, exposed to the internet via ngrok. That WebSocket feeds directly into a Gemini Live session, which acts as the AI patient, listening to the agent and responding in real time. A recorder sits alongside this live connection, capturing both sides of the conversation as it happens. The audio pipeline required format conversion at this layer: Gemini outputs PCM audio while Twilio expects mu-law, so resampling and encoding are handled before audio is returned to the call.

The architecture is modular, with distinct layers: the telephony layer manages Twilio call initialization and media stream handling, the LLM layer wraps Gemini Live for real-time conversational AI responses, and the services layer provides WebSocket server functionality and ngrok tunnel support for public webhook exposure.

Data Pipeline & Post-Call Analysis

Once the call ends, the recording moves through a two-stage post-call pipeline, both powered by Gemini. The first stage transcribes the full conversation from audio to text. That transcript is then passed back into Gemini a second time for bug analysis, evaluating the hospital agent's handling of the call against what a well-functioning patient intake conversation should look like. This post-call pipeline is intentionally decoupled from the live call path so that the processing load never touches the real-time conversation. The result is a closed-loop system: place the call, run the evaluation, record it, transcribe it, and surface issues, fully automated. The entire workflow is orchestrated at startup, with command-line scenario selection enabling rapid testing of different customer interaction patterns, and automatic call shutdown logic ensuring predictable test durations for consistent evaluation.

Results and my final findings

I ran ten scenarios. Six of them were flagged as significant issues after reviewing the bug analysis output. The most notable pattern: when the AI encountered a task it couldn't handle

cleanly, it would get stuck in a loop, repeatedly asking for the caller's name and date of birth, verifying those details over and over for sometimes close to two minutes, before finally escalating to a human agent. It was also prone to talking over the caller when things got difficult.

That's a real problem for a healthcare context. Knowing when to escalate quickly, rather than grinding through a verification loop, is critical when the person on the line might be anxious, in pain, or just trying to get help fast. The system surfaced that pattern clearly, which is exactly what it was built to do.

That's the project: an end-to-end automated evaluation harness for a voice AI, grounded in an actual understanding of what the business needed it to do well.